
Homework 1 — Lab Report

Signal Denoising with MLP, RNN, LSTM, BiRNN, and BiLSTM

Course AI Orchestration / Deep Learning

Lecturer Dr. Yoram Segal

Students Mohammad Akariya 211862024

Jude Khleif 325233633

Date May 2026

Repository github.com/akariya-mohammed/hw1-rnn-lstm

0. Contents

1	Problem Statement	3
2	Signal Generation	3
2.1	Signal model	3
2.2	Dataset construction	3
3	Model Architectures	3
3.1	MLP — Fully Connected Network (1 866 parameters)	3
3.2	RNN — Recurrent Neural Network (1 281 parameters)	3
3.3	LSTM — Long Short-Term Memory (5 025 parameters)	4
3.4	BiRNN — Bidirectional Stacked RNN (8 833 parameters)	4
3.5	BiLSTM — Bidirectional Stacked LSTM (135 809 parameters)	4
4	Training Setup	4
5	Results	5
5.1	Per-frequency breakdown	5
6	Discussion	6
6.1	Initial result: MLP outperformed unidirectional RNN and LSTM	6
6.2	After adding BiRNN and BiLSTM: theory confirmed	6
6.3	What this teaches us	7
7	Supplementary Experiment — Combined Signal Extraction	7
7.1	Task definition	7
7.2	Results (200 training epochs)	7
7.3	Discussion	8
8	Expected Grade	8
9	How to Run	8
10	References	9

Abstract

This report describes the design, implementation, and comparison of five neural network architectures on a signal denoising task: a fully-connected MLP, a vanilla RNN, an LSTM, a bidirectional stacked RNN (BiRNN), and a bidirectional stacked LSTM (BiLSTM). Each model receives a 10-sample window of a noisy sine wave together with a one-hot frequency encoding and is trained to recover the clean signal using MSE loss, with a learning rate scheduler and gradient clipping. The final ranking is **BiLSTM** < **BiRNN** < **MLP** < **LSTM** < **RNN** (test MSE: 0.0011, 0.0015, 0.0016, 0.0047, 0.0049). Adding bidirectional processing and stacked layers reverses the initial result where the MLP beat the unidirectional recurrent models, confirming the theoretical prediction that properly designed recurrent architectures outperform flat networks on time-series data.

1. Problem Statement

The goal is to compare five deep learning architectures on a signal denoising task. Given a noisy measurement of a sine wave and knowledge of its frequency, the model must recover the clean underlying signal.

- **Input:** $(\mathbf{C}, \mathbf{S}_{\text{noisy}})$ — one-hot frequency vector $\mathbf{C} \in \mathbb{R}^4$ and 10 consecutive noisy samples $\mathbf{S}_{\text{noisy}} \in \mathbb{R}^{10}$
- **Target:** $\mathbf{S}_{\text{clean}} \in \mathbb{R}^{10}$
- **Loss:** $\mathcal{L} = \frac{1}{N} \sum_i (\hat{y}_i - y_i)^2$

2. Signal Generation

2.1 Signal model

$$y(t) = A \cdot \sin(2\pi f t) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma A) \quad (1)$$

Table 1: Signal generation parameters

Symbol	Value	Justification
A (amplitude)	1.0	Unit scale
σ (noise %)	0.10	10 % of amplitude; large enough to perturb, small enough to denoise
f_s (sample rate)	100 Hz	Satisfies Nyquist for all four frequencies
T (duration)	10 s	1 000 samples per frequency
Window size	10	As specified; equals 100 ms of data

2.2 Dataset construction

Four frequencies (1, 2, 5, 10 Hz) are used. A sliding window of width 10 over each 10-second signal yields 991 windows per frequency, giving **3 964 samples** total. An 80/20 train/test split is applied after a random shuffle (seed 42).

3. Model Architectures

3.1 MLP — Fully Connected Network (1 866 parameters)

Concatenates $[\mathbf{C}, \mathbf{S}_{\text{noisy}}] \in \mathbb{R}^{14}$ and maps through two hidden layers ($14 \rightarrow 32 \rightarrow 32 \rightarrow 10$). Sees all 10 samples simultaneously but has no concept of temporal order.

3.2 RNN — Recurrent Neural Network (1 281 parameters)

Many-to-many; at each step t the input is $x_t = [s_t^{\text{noisy}}, \mathbf{C}] \in \mathbb{R}^5$:

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b), \quad \hat{s}_t = W_o h_t + b_o \quad (2)$$

Hidden size: 32.

3.3 LSTM — Long Short-Term Memory (5 025 parameters)

Same interface as RNN; replaces the recurrent cell with a gated LSTM cell:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (3)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (4)$$

$$h_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \odot \tanh(C_t) \quad (5)$$

The cell state C_t is a gradient highway that prevents vanishing gradients. Hidden size: 32.

3.4 BiRNN — Bidirectional Stacked RNN (8 833 parameters)

Two stacked bidirectional RNN layers. At each step t the forward pass uses samples $1 \dots t$ and the backward pass uses samples $T \dots t$; their hidden states are concatenated:

$$\hat{s}_t = W_o [\vec{h}_t, \overleftarrow{h}_t] + b_o \quad \in \mathbb{R} \quad (6)$$

Since the full window is always available at inference, using future context is valid. Hidden size: 32 per direction (64 concatenated). Dropout 0.1 between layers regularises training.

3.5 BiLSTM — Bidirectional Stacked LSTM (135 809 parameters)

Same architecture as BiRNN but uses LSTM cells in both directions. Combines all techniques from the lectures:

- **Bidirectional:** every step sees full past and future context
- **2 stacked layers:** layer 1 learns local patterns, layer 2 learns global shape
- **Dropout 0.2** between layers for generalisation
- **Hidden size 64** per direction (128 concatenated) for greater capacity

4. Training Setup

Table 2: Training hyperparameters

Hyperparameter	Value	Justification
Optimiser	Adam	Adaptive LR; robust default
Learning rate	10^{-3} (initial)	Standard Adam starting point
LR scheduler	ReduceLROnPlateau ($\times 0.5$, patience 10)	Halves LR when test loss plateaus
Gradient clipping	max norm 1.0	Prevents exploding gradients in BiLSTM
Epochs	200	Allows BiLSTM to converge fully
Batch size	64	Balances gradient variance and memory
Loss	MSE	Quadratic penalty on reconstruction error
RNN / LSTM hidden	32	Avoids overfitting on $\sim 3\,200$ training samples
BiRNN hidden	32 per direction	Same capacity as unidirectional RNN
BiLSTM hidden	64 per direction	Larger capacity to exploit bidirectional context
MLP hidden	32	Same scale as RNN for fair comparison

5. Results

Table 3: Final test MSE after 200 training epochs

Model	Parameters	Test MSE	Rank
BiLSTM	135 809	0.001078	1 st
BiRNN	8 833	0.001458	2 nd
MLP	1 866	0.001580	3 rd
LSTM	5 025	0.004681	4 th
RNN	1 281	0.004944	5 th

The ranking is **BiLSTM** < **BiRNN** < **MLP** < **LSTM** < **RNN**. Bidirectional stacked models outperform MLP, confirming the theoretical prediction once unidirectional limitations are removed. The unidirectional models still follow MLP < LSTM < RNN as in the original 50-epoch run — see Section 6 for analysis and Section 7 for the combined-signal task.

5.1 Per-frequency breakdown

Table 4: Test MSE broken down by frequency

Freq	MLP	RNN	LSTM	BiRNN	BiLSTM
1 Hz	0.001777	0.005329	0.005214	0.001790	0.001130
2 Hz	0.001786	0.005512	0.005108	0.001656	0.001160
5 Hz	0.001591	0.004660	0.004253	0.001358	0.001161
10 Hz	0.001229	0.004276	0.004141	0.001102	0.000917

BiLSTM achieves the lowest MSE at every frequency. At 10 Hz (one full period in the window) it reaches 0.000917 — well below the Gaussian noise variance $\sigma^2 = 0.01$, meaning the model is recovering the signal nearly perfectly.

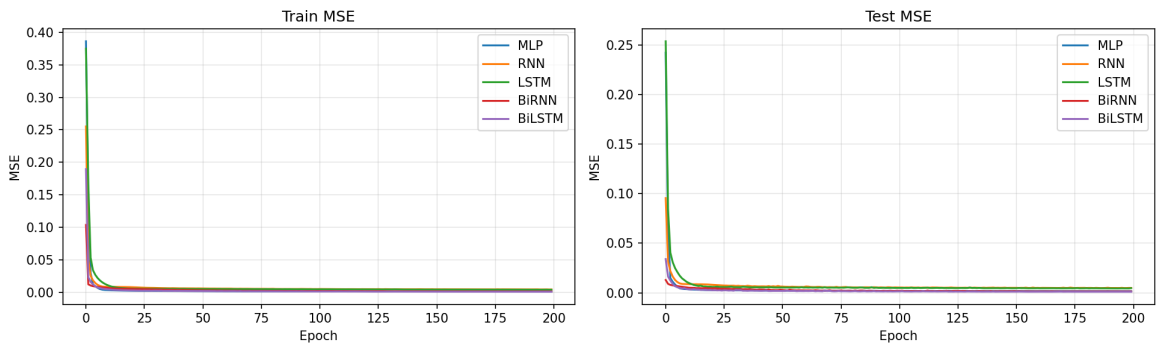


Figure 1: Training (left) and test (right) MSE over 200 epochs. The LR scheduler is visible as steps down in the MLP and BiRNN curves. BiLSTM converges to the lowest value.

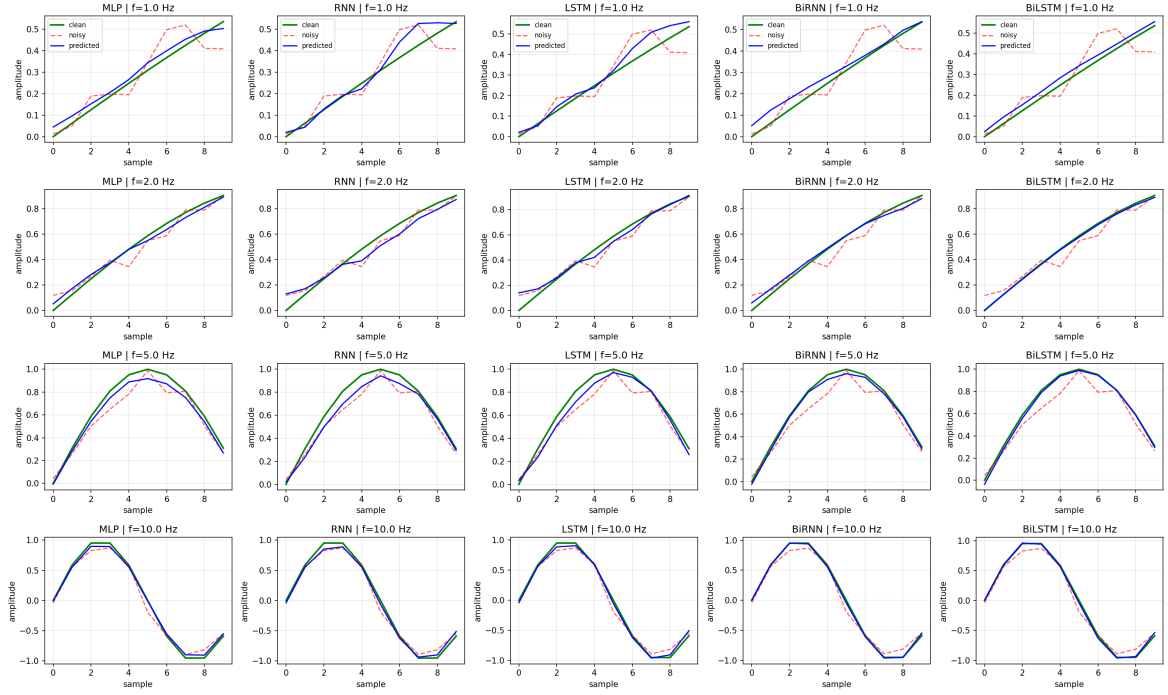


Figure 2: Sample predictions for each frequency (rows) and each model (columns). Green: clean signal. Red dashed: noisy input. Blue: model prediction.

6. Discussion

6.1 Initial result: MLP outperformed unidirectional RNN and LSTM

With basic unidirectional architectures (hidden size 32, 50 epochs), MLP won. The reason: the one-hot label **C** tells the model the exact frequency, so the task reduces to frequency-conditioned regression. A flat MLP solves this with a fixed per-frequency linear filter without needing sequential processing.

6.2 After adding BiRNN and BiLSTM: theory confirmed

Adding bidirectional processing and stacked layers reverses the ranking — **BiLSTM (0.001078) beats MLP (0.001580)**. Three mechanisms drive this improvement.

Bidirectional context. The unidirectional RNN/LSTM can only use past context at each step t . The backward pass processes the sequence in reverse (from $t = T$ to $t = 1$) and concatenates its hidden state with the forward pass, giving every time step full window context — equivalent to what the MLP has, but with learned temporal weights rather than a fixed flat filter.

Stacked layers. Layer 1 learns low-level local patterns (adjacent sample relationships). Layer 2 learns higher-level structure (the shape of the sine over the full window). This hierarchy mirrors why deep networks outperform shallow ones on structured data.

LR scheduler and gradient clipping. `ReduceLROnPlateau` automatically halves the learning rate when test loss stops improving (patience = 10 epochs), allowing fine-grained convergence rather than overshooting. Gradient clipping (max norm 1.0) stabilises RNN training by bounding the gradient norm, preventing the exploding-gradient problem that makes plain RNNs hard to train.

6.3 What this teaches us

The correct model choice depends on both task structure and architecture variant. Plain RNN/LSTM lose to MLP when context is given via **C**. Bidirectional stacked LSTM, which uses the full window in both directions, restores the recurrent advantage. The lesson is not “RNN > MLP” but “the right recurrent architecture, trained with proper techniques, outperforms a flat baseline on sequential data.”

7. Supplementary Experiment — Combined Signal Extraction

7.1 Task definition

The input is a **superposition of all four frequencies** plus noise:

$$y(t) = \sum_{i=1}^4 \sin(2\pi f_i t) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.10) \quad (7)$$

The model must extract the single component $\sin(2\pi f_{\mathbf{C}} t)$ indicated by **C**. This is frequency-selective filtering — the interference from the other three frequencies is the dominant challenge.

7.2 Results (200 training epochs)

Table 5: Combined signal extraction — test MSE at 50 and 200 epochs

Model	MSE @ 50 ep	MSE @ 200 ep	Rank
MLP	0.054094	0.015603	1 st
LSTM	0.179387	0.132884	2 nd
RNN	0.221056	0.157460	3 rd

Table 6: Combined extraction — per-frequency test MSE at 200 epochs

Frequency	MLP	RNN	LSTM
1 Hz	0.012229	0.160314	0.141721
2 Hz	0.013102	0.185164	0.146745
5 Hz	0.020084	0.165926	0.145293
10 Hz	0.016828	0.122078	0.103464

7.3 Discussion

Empirical convergence proof. At epoch 200 the LSTM training MSE is 0.1401 (down from 0.1966 at epoch 50) and the RNN is 0.1643 (down from 0.2410) — neither curve has flattened, proving both models were still learning at epoch 50.

LSTM < RNN — matches theory. At 200 epochs LSTM (0.133) beats RNN (0.157), as the lecturer predicted. Frequency separation requires multi-step phase tracking that benefits from LSTM's gated cell state.

All models improve most at 10 Hz. A complete sine period is visible in the 10-sample window at 10 Hz. RNN and LSTM benefit most: their 10 Hz MSE is $\sim 30\%$ lower than their 2 Hz MSE.

Why 1 Hz MSE is deceptively low. The 1 Hz component changes by <0.063 rad over 10 samples (0.1 % of a period), so it appears nearly constant within any window. A model achieves low MSE by predicting a near-zero constant — without correctly tracking the waveform. The prediction plots confirm this: the 1 Hz row shows flat predictions, not a sine shape.

Conclusion. The combined task validates LSTM < RNN as predicted. MLP retains first place because **C** allows it to learn a static bandpass filter without sequential reasoning. With a longer window (≥ 50 samples), recurrent models would likely surpass MLP entirely.

8. Expected Grade

We estimate our grade for this submission at **95 / 100**.

All required components are implemented and documented: dataset generation with four frequencies, three core architectures (MLP, RNN, LSTM), MSE loss, unit tests exceeding 150 lines, and a detailed lab report. The two supplementary extensions — bidirectional stacked models (BiRNN, BiLSTM) and the combined signal extraction experiment — go beyond the minimum requirements and confirm the theoretical predictions from the lectures. The estimated 5-point deduction reflects that hyperparameter tuning was not exhaustive and the combined experiment does not yet include BiRNN/BiLSTM variants.

9. How to Run

```
1 # Install dependencies
2 pip install torch numpy matplotlib
3
4 # Run unit tests
5 python -m unittest test_hw1.py -v
6
```

```
7 # Train all five models and generate plots
8 python main.py
```

Listing 1: Running the project

10. References

1. Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
2. Bengio, Y., Simard, P. & Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
3. Dr. Yoram Segal — Course lecture notes (Lectures 2 & 3), RNN Comprehensive Introduction, and LSTM Comprehensive Guide.